
🔴 **SAVOL 36:**

`JUPYTER NOTEBOOK` NING MA'LUMOT TAHLILIDA TUTGAN O'RNI. NIMA UCHUN ML LOYIHALARIDA ODATIY PYTHON SKRIPTIDAN KO'RA NOTEBOOK AFZAL? INTERAKTIV TAHLIL IMKONIYATLARI.

---

1. Jupyter Notebook — Nima Va Nima Uchun Yaratilgan?

```
Tarix:
  2011 → IPython Notebook sifatida boshlangan
  2014 → Jupyter nomi bilan mustaqil loyiha
  "Jupyter" = Julia + Python + R (qo'llab-quvvatlangan tillar)
  2018 → JupyterLab (yangi interfeys)

Maqsad:
  "Literate Programming" → Donald Knuth (1984)
  Kod + Hujjat + Natija → BITTA joyda

  Olimlar va tahlilchilar uchun:
  "Men nima qildim va nima topдim?" →
  Kod, grafik, izoh — hammasi bitta faylda
```

Jupyter ning markaziy g'oyasi:

```
Odatiy Python skript:
  kod.py → Ishga tushirish → Natija (terminal)
  → Natija ko'rinmaydi (print bo'lmasa)
  → Grafik alohida oyna
  → Izoh → Faqat # bilan

Jupyter Notebook:
  .ipynb → Cell lar ketma-ketligi
  → Har cell: Kod, Natija, Grafik → BIRGALIKDA
  → Markdown → Chiroyli izoh, formula, rasm
  → Interaktiv → Bir cell o'zgartirilsa, boshqalar saqlanib qoladi
```

---

2. Notebook Tuzilishi — Cell Turlari

```
Jupyter Notebook = .ipynb fayl (JSON format)

Cell turlari:

1. Code Cell:
   → Python kodi yoziladi va bajariladi
   → In [1]: kod
     Out[1]: natija

2. Markdown Cell:
```

```
   → Formatlangan matn
   → # Sarlavha,  **qalin**,  *kursiv*
   → LaTeX formulalar: $y = \beta_0 + \beta_1 x$
   → Jadval, ro'yxat, rasm

3. Raw Cell:
   → Hech qanday qayta ishlash yo'q
   → nbconvert uchun (PDF, HTML ga o'tkazish)
```

Execution Order — Muhim Tushuncha:

```
Notebook da cell lar ISTALGAN tartibda bajariladil!

In [1]: x = 10
In [2]: y = x + 5   → y = 15
In [3]: x = 20
In [4]: print(y)    → 15 (!) → Xato kutilishi mumkin edi

Nima bo'ldi?
  [2] da x=10 edi → y=15 hisoblandi → Saqlanib qoldi
  [3] da x=20 ga o'zgartirildi
  [4] da y hali 15 → Chunki [2] qayta bajarilmadi

Muammo:
  Tartibsiz bajarish → Qayta ishlab bo'lmaydigan natijalar
  "Restart & Run All" → Tartibli bajarish kafolati

Qoida:
  Tayyor notebook → Yuqoridan pastga to'liq ishlashi kerak!
```

---

3. Odatiy Python Skript vs Jupyter Notebook

Odatiy Skript afzalliklari:

```
✅ Version control (Git) → Aniq diff ko'rsatiladi
✅ Production deployment → Server da ishga tushirish
✅ Import qilish mumkin   → Boshqa modulda ishlatish
✅ Avtomatlashtirish      → Cron job, CI/CD
✅ Katta loyihalar        → Modulli tuzilma
✅ IDE qo'llab-quvvatlash → Debugging oson
✅ Refactoring            → Kodni tartibga keltirish
```

Jupyter Notebook afzalliklari:

```
✅ Interaktiv tahlil      → Har qadam natijasi darhol
✅ Eksperiment            → Tez sinash, o'zgartirish
✅ Vizualizatsiya         → Grafik to'g'ridan-to'g'ri
✅ Hikoya qilish          → Kod + izoh + natija
✅ Kuzatuvlar saqlash     → Avvalgi natijalar yo'qolmaydi
✅ Ta'lim va demo         → Tushuntirish oson
✅ EDA (Tahlil)           → Ma'lumotni o'rganish
✅ Hamkorlik              → nbviewer, Google Colab
```

Qachon qaysi:

```
Jupyter Notebook:
  → Ma'lumotni dastlabki o'rganish (EDA)
  → Eksperiment, prototiplash
  → Natijalarni taqdim etish
  → Ta'lim, o'rgatish
  → Bir martalik tahlil
  → Ilmiy tadqiqot

Python Skript:
  → Production model deployment
  → Kutubxona/paket yaratish
  → Avtomatlashtirilgan pipeline
  → Katta muhandislik loyihasi
  → Test yozish (pytest)
  → API server (FastAPI, Flask)
```

---

4. ML Loyihasida Notebook Afzalliklari

Afzallik 1 — EDA (Exploratory Data Analysis):

```python
# Skriptda:
import pandas as pd
df = pd.read_csv("data.csv")
print(df.head())
print(df.describe())
print(df.isnull().sum())
# → Hammasi terminalda, chalkash!

# Notebookda:
df.head()        # Chiroyli jadval ko'rinishida
                 # ↓ (Natija shu yerda ko'rinadi)
```

	Yosh	Daromad	Kasb
0	35	50000	Muhandis
1	28	35000	Dizayner

Afzallik 2 — Interaktiv Grafik:

```python
# Notebookda grafik to'g'ridan-to'g'ri ko'rinadi:
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].hist(df["yosh"], bins=30, color="steelblue")
axes[0].set_title("Yosh taqsimoti")

sns.boxplot(data=df, x="kasb", y="daromad", ax=axes[1])
axes[1].set_title("Kasb vs Daromad")
```

```python
sns.heatmap(df.corr(), annot=True, ax=axes[2])
axes[2].set_title("Korrelyatsiya")

plt.tight_layout()
plt.show()
# → Grafik notebook ichida paydo bo'ladi!
```

Afzallik 3 — Bosqichma-bosqich Eksperiment:

```python
# Cell 1: Ma'lumot yuklash
df = pd.read_csv("data.csv")
print(f"O'lcham: {df.shape}")

# Cell 2: Tozalash (faqat shu cell bajariladi)
df_clean = df.dropna()
df_clean = df_clean[df_clean["yosh"] > 0]
print(f"Tozalangandan keyin: {df_clean.shape}")

# Cell 3: Feature engineering (oldingi natijalar saqlanib qolgan!)
df_clean["log_daromad"] = np.log(df_clean["daromad"])
# → df_clean ni qayta yuklash shart emas!

# Cell 4: Model (faqat model qismini o'zgartirish mumkin)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
# → Ma'lumot preprocessingi qayta bajarilmaydi
```

Afzallik 4 — Kuzatuvlar Saqlash:

```
Skriptda:
  Kod o'zgartirildi → Avvalgi natija YO'QOLDI
  "Kecha qanday natija chiquvdi?" → Bilmaymiz!

Notebookda:
  Har cell ning natijasi SAQLANIB QOLADI
  Notebook yopilsa ham natija .ipynb faylida

  In [42]: model.score(X_test, y_test)
  Out[42]: 0.8734   ← Hali ham ko'rinib turibdi!
```

Afzallik 5 — Narrative (Hikoya):

```
## 3. Model O'rgatish

Yuqoridagi EDA dan ko'ringanidek, `daromad` va `tajriba`
orasida kuchli korrelyatsiya mavjud (r=0.72).

Shuning uchun **Random Forest** ni tanladik:
- Nochiziqli munosabatlarni ushlab oladi
- Feature importance beradi
- Overfittingga chidamli

Natijalar quyida ko'rsatilgan:
# Kod cell
rf = RandomForestRegressor(n_estimators=100)
```

```
rf.fit(X_train, y_train)
print(f"R²: {rf.score(X_test, y_test):.4f}")

    R²: 0.8734

Model **87.3%** ni tushuntiradi — qoniqarli natija.
Keyingi qadamda giperparametrlarni sozlaymiz.
```

---

5. Jupyter Ning Interaktiv Imkoniyatlari

Magic Commands:

```
# Vaqt o'lchash:
%time model.fit(X_train, y_train)
# CPU times: user 2.3 s, sys: 0.1 s, total: 2.4 s

%%time
# Butun cell vaqti
for i in range(1000):
    pass

# Fayl o'qish/yozish:
%%writefile utils.py
def hello():
    return "Salom!"

%run utils.py   # Faylni ishga tushirish

# Matplotlib inline:
%matplotlib inline   # Grafik notebook ichida

# Avtomatik qayta yuklash:
%load_ext autoreload
%autoreload 2   # Import qilingan modul o'zgarsa qayta yuklanadi
```

ipywidgets — Interaktiv Boshqaruv:

```
from ipywidgets import interact, IntSlider, Dropdown
import matplotlib.pyplot as plt
import numpy as np

# Slider bilan interaktiv grafik:
def grafik_korsot(daraja, namunalar):
    x = np.linspace(-3, 3, namunalar)
    y = x ** daraja
    plt.figure(figsize=(8, 4))
    plt.plot(x, y, "b-", linewidth=2)
    plt.title(f"y = x^{daraja}")
    plt.grid(True)
    plt.show()

interact(
    grafik_korsot,
    daraja=IntSlider(min=1, max=5, value=2),
```

```
    namunalar=IntSlider(min=10, max=200, value=100)
)
# → Slider qo'shimchasi → Grafik darhol yangilanadi!
```

Tqdm — Progress Bar:

```
from tqdm.notebook import tqdm
import time

# Notebook uchun chiroyli progress bar:
natijalar = []
for k in tqdm(range(1, 11), desc="K qiymatlari"):
    model = KMeans(n_clusters=k)
    model.fit(X)
    natijalar.append(model.inertia_)

# → [=====>    ] 5/10 50% | 0:00:03<0:00:03
```

---

6. Notebook Strukturasi — ML Loyihasi

```
Yaxshi tuzilgan ML Notebook:

# 1. Sarlavha va Maqsad
  ## Uy Narxini Bashorat Qilish
  **Maqsad:** ...  **Ma'lumot:** ...  **Metodlar:** ...

# 2. Kutubxonalar
  import pandas as pd
  import numpy as np
  ...

# 3. Ma'lumot Yuklash
  df = pd.read_csv(...)
  df.head(), df.info(), df.describe()

# 4. EDA (Exploratory Data Analysis)
  Taqsimotlar, korrelyatsiya, anomaliyalar

# 5. Ma'lumot Tozalash
  Bo'sh qiymatlar, dublikatlar, outlierlar

# 6. Feature Engineering
  Yangi xususiyatlar, transformatsiyalar

# 7. Model Tanlash va O'rgatish
  Train-test, Pipeline, CV

# 8. Baholash
  Metrikalar, grafik, taqqoslash

# 9. Xulosa
  Asosiy topilmalar, keyingi qadamlar
```

---

7. Notebook Ning Kamchiliklari Va Yechimlar

```
Kamchilik 1 — Tartibsiz bajarish:
  Yechim: "Restart & Run All" → Har doim
  Yechim: Nomerlangan cell lar tartibda

Kamchilik 2 — Version Control (Git):
  .ipynb = JSON → Git diff chalkash
  Yechim: nbstripout → Output larni o'chirib saqlash
  Yechim: Jupytext → .py va .ipynb sinxronlashtirish

Kamchilik 3 — Katta loyihalarda chalkashlik:
  1000+ qatorli notebook → Boshqarib bo'lmaydi
  Yechim: Modullar → .py fayllar → Notebookda import
  Yechim: Papermill → Notebook ni parametrli ishga tushirish

Kamchilik 4 — Debugging qiyin:
  Yechim: %debug magic command
  Yechim: pdb breakpoint
  Yechim: VS Code Jupyter extension

Kamchilik 5 — Resurs boshqaruvi:
  Katta ma'lumot → Xotira to'lib qolishi
  Yechim: del df → Xotirani bo'shatish
  Yechim: Chunked reading → pd.read_csv(chunksize=)
```

---

8. Google Colab — Bulutli Jupyter

```
Google Colab = Jupyter Notebook + Bepul GPU/TPU

Afzalliklari:
  ✅ O'rnatish shart emas → Brauzerda ishlaydi
  ✅ Bepul GPU (Tesla T4) → Chuqur o'rganish uchun
  ✅ Google Drive integratsiyasi
  ✅ Hamkorlik → Bir vaqtda bir nechta odam
  ✅ Snippet kutubxonasi

Farqlari:
  → Sessiya tugasa → O'zgaruvchilar yo'qoladi
  → Bepul versiyada vaqt cheklangan (12 soat)
  → Ma'lumot saqlanmaydi (Drive ga yuklash kerak)

from google.colab import drive
drive.mount("/content/drive")
df = pd.read_csv("/content/drive/MyDrive/data.csv")
```

---

9. Xulosa

```
Jupyter Notebook:
  Kod + Natija + Grafik + Izoh → BITTA joyda
  .ipynb → JSON format
```

```
Odatiy Skript vs Notebook:
  Skript   → Production, deployment, katta loyiha
  Notebook → EDA, eksperiment, taqdimot, ta'lim

ML Loyihasida Afzalliklari:
  ✅ Bosqichma-bosqich tahlil
  ✅ Natijalar saqlanib qoladi
  ✅ Grafik inline ko'rsatiladi
  ✅ Markdown → Hikoya qilish
  ✅ Interaktiv widgets
  ✅ Magic commands (%time, %%timeit)

Muhim Qoidalar:
  ✅ "Restart & Run All" → Har doim ishlashi kerak
  ✅ Yuqoridan pastga mantiqiy tartib
  ✅ Katta funksiyalar → .py faylga, Notebookda import
  ✅ nbstripout → Git da output larni tozalash

Execution Order:
  → Tasodifiy tartibda bajarish → Xatoga olib keladi
  → Doim yuqoridan pastga tartib → Professional yondashuv
```

> 💡 **Eslab qol:** Jupyter Notebook — **laboratoriya daftari** kabidir. Olim tajriba o'tkazganda har kuzatuvni, natijani, izohni daftarga yozadi. Notebook ham shunday — har qadam izchil saqlanadi. Lekin "Restart & Run All" ishlash kerak qoidasi buzilsa — daftar sahifalari aralashib ketgan kabi: natijalar ishonchsiz bo'ladi!

🔴 **SAVOL 37:**

`TITANIC` YOKI `IRIS` KABI MASHHUR DATASETLARNI TAHLIL QILISH JARAYONI: `EDA` (`EXPLORATORY DATA ANALYSIS`) BOSQICHLARI, MUHIM XUSUSIYATLARNI ANIQLASH VA DASTLABKI MODEL QURISH.

---

1. EDA Nima Va Nima Uchun Kerak?

```
EDA — Ma'lumotni modellashtirish OLDIDAN chuqur o'rganish

John Tukey (1977):
  "EDA — ma'lumotga savol berish san'ati.
   Model qurishdan avval ma'lumot nima demoqchi?"

EDA maqsadlari:
  → Ma'lumot tuzilishini tushunish
  → Anomaliya va shovqinlarni aniqlash
  → O'zgaruvchilar orasidagi munosabatlarni topish
  → Model uchun gipoteza shakllantirish
  → Feature Engineering uchun asos yaratish

EDA siz model qurish:
  → Ko'r bo'lib piyoda yurish kabi
```

```
→ Muhim naqshlar o'tkazib yuboriladi
→ Noto'g'ri modellar tanlanadi
```

---

2. EDA Bosqichlari — Tizimli Yondashuv

```
Bosqich 1: Ma'lumotni yuklash va dastlabki ko'rish
Bosqich 2: O'lcham va tur tekshirish
Bosqich 3: Bo'sh qiymatlar tahlili
Bosqich 4: Statistik tavsif
Bosqich 5: Univariat tahlil (bitta o'zgaruvchi)
Bosqich 6: Bivariat tahlil (ikki o'zgaruvchi)
Bosqich 7: Multivariat tahlil (ko'p o'zgaruvchi)
Bosqich 8: Anomaliyalar (Outlierlar)
Bosqich 9: Xulosa va gipotezalar
```

---

3. Titanic Dataset — Kontekst

```
Titanic (1912):
  2,224 yo'lovchi va ekipaj
  1,502 ta halok bo'ldi (67.5%)

Dataset:
  891 ta kuzatuv (train)
  12 ta o'zgaruvchi

O'zgaruvchilar:
  Survived    → Maqsad (0=Halok, 1=Omon)
  Pclass      → Bilet sinfi (1=Birinchi, 2=Ikkinchi, 3=Uchinchi)
  Name        → Ism
  Sex         → Jins
  Age         → Yosh
  SibSp       → Aka-uka/opa-singil soni
  Parch       → Ota-ona/bola soni
  Ticket      → Bilet raqami
  Fare        → Bilet narxi
  Cabin       → Kabina
  Embarked    → Chiqish porti (C=Cherbourg, Q=Queenstown, S=Southampton)
```

---

4. Bosqich 1-4: Dastlabki Ko'rish

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv("titanic.csv")

# O'lcham:
print(df.shape)           # (891, 12)
```

```
# Dastlabki qatorlar:
df.head()

# Tur va bo'sh qiymatlar:
df.info()
# RangeIndex: 891 entries
# Age      714 non-null  → 177 ta bo'sh!
# Cabin    204 non-null  → 687 ta bo'sh (77%)!
# Embarked 889 non-null  → 2 ta bo'sh

# Statistik tavsif:
df.describe()
# count  mean   std    min   25%   50%   75%   max
# Age    714  29.7   14.5   0.42  20   28    38    80
# Fare   891  32.2   49.7   0     7.9  14.5  31.0  512

# Maqsad o'zgaruvchi taqsimoti:
print(df["Survived"].value_counts(normalize=True))
# 0    0.616  → 61.6% halok bo'lgan
# 1    0.384  → 38.4% omon qolgan
# → Imbalanced! Accuracy yolg'on bo'lishi mumkin
```

---

5. Bosqich 5: Univariat Tahlil

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Yosh taqsimoti:
axes[0,0].hist(df["Age"].dropna(), bins=30,
               color="steelblue", edgecolor="white")
axes[0,0].set_title("Yosh Taqsimoti")
axes[0,0].axvline(df["Age"].mean(), color="red",
                  linestyle="--", label=f"O'rtacha: {df['Age'].mean():.1f}")
axes[0,0].legend()

# Narx taqsimoti (qiyroq!):
axes[0,1].hist(df["Fare"], bins=50,
               color="coral", edgecolor="white")
axes[0,1].set_title("Bilet Narxi (Qiyroq!)")

# Log transformatsiya kerakmi?
axes[0,2].hist(np.log1p(df["Fare"]), bins=30,
               color="green", edgecolor="white")
axes[0,2].set_title("Log(Narx) — Normalroq")

# Sinf taqsimoti:
df["Pclass"].value_counts().plot(kind="bar",
    ax=axes[1,0], color=["gold","silver","brown"])
axes[1,0].set_title("Bilet Sinfi")

# Jins taqsimoti:
df["Sex"].value_counts().plot(kind="pie",
    ax=axes[1,1], autopct="%1.1f%%",
    colors=["steelblue","coral"])
axes[1,1].set_title("Jins")
```

```python
# Chiqish porti:
df["Embarked"].value_counts().plot(kind="bar",
    ax=axes[1,2], color="purple")
axes[1,2].set_title("Chiqish Porti")

plt.tight_layout()
plt.show()
```

Univariat topilmalar:

```
    Yosh:
      → O'rtacha: 29.7, Mediana: 28
      → Yosh taqsimoti biroz o'ng tomonga og'gan
      → Bolalar (0-10) alohida guruh ko'rinadi

    Bilet narxi:
      → Kuchli qiyroq taqsimot (o'rtacha 32, max 512!)
      → Log transformatsiya kerak

    Sinf:
      → 3-sinf: 55%, 1-sinf: 24%, 2-sinf: 21%
      → Ko'pchilik uchinchi sinfda

    Jins:
      → Erkak: 65%, Ayol: 35%
```

---

6. Bosqich 6: Bivariat Tahlil — Eng Muhim Qism

```python
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# 1. Jins vs Omon qolish:
survival_sex = df.groupby("Sex")["Survived"].mean()
survival_sex.plot(kind="bar", ax=axes[0,0],
    color=["steelblue","coral"], rot=0)
axes[0,0].set_title("Jins → Omon qolish ehtimoli")
axes[0,0].set_ylabel("Omon qolish %")
axes[0,0].set_ylim(0, 1)
for i, v in enumerate(survival_sex):
    axes[0,0].text(i, v+0.02, f"{v:.1%}", ha="center")
# Ayol: 74.2%, Erkak: 18.9% → KATTA FARQ!

# 2. Sinf vs Omon qolish:
survival_pclass = df.groupby("Pclass")["Survived"].mean()
survival_pclass.plot(kind="bar", ax=axes[0,1],
    color=["gold","silver","brown"], rot=0)
axes[0,1].set_title("Bilet Sinfi → Omon qolish")
# 1-sinf: 63%, 2-sinf: 47%, 3-sinf: 24%

# 3. Yosh vs Omon qolish (KDE):
df[df["Survived"]==1]["Age"].dropna().plot(
    kind="kde", ax=axes[0,2], label="Omon", color="green")
df[df["Survived"]==0]["Age"].dropna().plot(
    kind="kde", ax=axes[0,2], label="Halok", color="red")
axes[0,2].set_title("Yosh → Omon qolish")
```

```
axes[0,2].legend()
# Bolalar (0-10) ko'proq omon qolgan!

# 4. Narx vs Omon qolish (Boxplot):
df.boxplot(column="Fare", by="Survived", ax=axes[1,0])
axes[1,0].set_title("Narx vs Omon qolish")
# Omon qolganlar ko'proq pul to'lagan → 1-sinf

# 5. Oila a'zolari vs Omon qolish:
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
survival_family = df.groupby("FamilySize")["Survived"].mean()
survival_family.plot(kind="bar", ax=axes[1,1], color="purple")
axes[1,1].set_title("Oila O'lchami → Omon qolish")
# Yolg'iz: 30%, Kichik oila(2-4): 55-60%, Katta oila: 20%

# 6. Sinf + Jins kombinatsiyasi:
pivot = df.pivot_table(values="Survived",
    index="Sex", columns="Pclass", aggfunc="mean")
sns.heatmap(pivot, annot=True, fmt=".2f",
    cmap="YlOrRd", ax=axes[1,2])
axes[1,2].set_title("Sinf × Jins → Omon qolish")

plt.tight_layout()
plt.show()
```

Bivariat topilmalar — Gipotezalar:

```
G'oya 1 — "Ayollar va bolalar birinchi" qoidasi:
  Ayol: 74% vs Erkak: 19% → ✅ Tasdiqlandi!
  Bolalar (< 10): ~58% → ✅ Tasdiqlandi!

G'oya 2 — Boylar ustunlikka ega:
  1-sinf: 63% vs 3-sinf: 24% → ✅ Tasdiqlandi!
  Narx bilan kuchli korrelyatsiya

G'oya 3 — Oila o'lchami muhim:
  Yolg'iz: 30% vs Kichik oila: 55% →
  Katta oila (7+): 13% → ✅ Qisman tasdiqlandi
  "U-shakl" munosabat

G'oya 4 — Kombinatsiya eng kuchli:
  1-sinf Ayol: 97%! (deyarli hammasi omon)
  3-sinf Erkak: 14%  (deyarli hammasi halok)
  → Sinf + Jins → Eng muhim kombinatsiya!
```

---

7. Bosqich 7: Multivariat Tahlil

```
# Korrelyatsiya matritsasi (raqamli ustunlar):
raqamli = df[["Survived","Pclass","Age",
              "SibSp","Parch","Fare","FamilySize"]].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(
    raqamli,
    annot=True, fmt=".2f",
```

```
    cmap="coolwarm", center=0,
    vmin=-1, vmax=1
)
plt.title("Korrelyatsiya Matritsasi")
plt.show()

# Pairplot — Barcha juftlar:
sns.pairplot(
    df[["Survived","Pclass","Age","Fare"]].dropna(),
    hue="Survived",
    palette={0:"red", 1:"green"},
    diag_kind="kde",
    plot_kws={"alpha": 0.5}
)
plt.show()
```

Korrelyatsiya topilmalari:

```
    Survived bilan korrelyatsiya:
      Pclass:    -0.34  → Sinf oshsa omon qolish kamayadi
      Fare:      +0.26  → Narx oshsa omon qolish oshadi
      Age:       -0.08  → Zaif manfiy
      FamilySize: +0.02 → Deyarli yo'q (nochiziqli!)

    Multikolinearlik:
      Pclass ↔ Fare:  -0.55 → O'rtacha korrelyatsiya
      SibSp ↔ Parch:  +0.41 → FamilySize bilan almashtirish mumkin
```

---

8. Bosqich 8: Anomaliyalar

```
# IQR usuli:
Q1 = df["Fare"].quantile(0.25)
Q3 = df["Fare"].quantile(0.75)
IQR = Q3 - Q1

pastki = Q1 - 1.5 * IQR
yuqori = Q3 + 1.5 * IQR

outlierlar = df[(df["Fare"] < pastki) |
                (df["Fare"] > yuqori)]
print(f"Outlierlar: {len(outlierlar)} ta")
print(f"Max Fare: {df['Fare'].max()}")  # 512 → Juda katta!

# Z-score usuli:
from scipy import stats
z_scores = np.abs(stats.zscore(df["Age"].dropna()))
print(f"Yosh outlierlar (|z|>3): {(z_scores > 3).sum()} ta")
```

---

9. Feature Engineering — EDA Topilmalari Asosida

```
def titanic_features(df):

    # 1. Jins → Raqam
```

```
    df["Sex_num"] = (df["Sex"] == "female").astype(int)

    # 2. Yosh bo'sh qiymatlarini to'ldirish
    df["Age_filled"] = df["Age"].fillna(df["Age"].median())

    # 3. Yosh guruhlari (EDA dan: bolalar muhim!)
    df["Age_group"] = pd.cut(
        df["Age_filled"],
        bins=[0, 12, 18, 35, 60, 100],
        labels=["Bola","O'smir","Yosh","O'rta","Katta"]
    )

    # 4. Oila o'lchami (EDA dan: U-shakl munosabat)
    df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

    # 5. Yolg'iz bayrog'i
    df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

    # 6. Narxni log ga o'tkazish (EDA dan: qiyroq)
    df["Log_Fare"] = np.log1p(df["Fare"])

    # 7. Unvon (Ismdan ajratib olish)
    df["Title"] = df["Name"].str.extract(r",\s*([^.]+)\.")
    rare = ["Dr","Rev","Col","Major","Mme","Countess",
            "Capt","Lady","Sir","Mlle","Don","Jonkheer"]
    df["Title"] = df["Title"].replace(rare, "Rare")
    df["Title"] = df["Title"].replace(
        {"Miss":"Miss","Ms":"Miss","Mrs":"Mrs","Mr":"Mr"})

    # 8. Unvon → Raqam
    title_map = {"Mr":0,"Miss":1,"Mrs":2,"Master":3,"Rare":4}
    df["Title_num"] = df["Title"].map(title_map)

    # 9. Chiqish porti bo'sh qiymatlar
    df["Embarked"] = df["Embarked"].fillna("S")
    embarked_map = {"S":0, "C":1, "Q":2}
    df["Embarked_num"] = df["Embarked"].map(embarked_map)

    return df

df = titanic_features(df)
```

---

10. Dastlabki Model Qurish

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

# Xususiyatlar tanlash (EDA asosida):
xususiyatlar = [
    "Pclass",        # Kuchli (r=-0.34)
    "Sex_num",       # Eng kuchli
```

```
    "Age_filled",    # Muhim (bolalar)
    "Log_Fare",      # O'rtacha
    "FamilySize",    # U-shakl
    "IsAlone",       # Qo'shimcha signal
    "Title_num",     # Jins+yosh kombinatsiyasi
    "Embarked_num"   # Qo'shimcha
]

X = df[xususiyatlar]
y = df["Survived"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Model 1 — Logistik Regressiya (Baseline):
lr_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model",  LogisticRegression(random_state=42))
])

# Model 2 — Random Forest:
rf_pipe = Pipeline([
    ("model", RandomForestClassifier(
        n_estimators=100, random_state=42))
])

# Cross-validation:
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for nom, pipe in [("Logistik", lr_pipe), ("RandomForest", rf_pipe)]:
    skorlar = cross_val_score(pipe, X_train, y_train,
                              cv=cv, scoring="accuracy")
    print(f"{nom}: {skorlar.mean():.4f} ± {skorlar.std():.4f}")

# RandomForest: 0.8372 ± 0.0189 ✅

# Final model:
rf_pipe.fit(X_train, y_train)
y_pred = rf_pipe.predict(X_test)
print(classification_report(y_test, y_pred))
```

Feature Importance — EDA ni Tasdiqlash:

```
rf_model = rf_pipe.named_steps["model"]
importance = pd.Series(
    rf_model.feature_importances_,
    index=xususiyatlar
).sort_values(ascending=False)

importance.plot(kind="barh", figsize=(8, 5), color="steelblue")
plt.title("Feature Importance")
plt.show()

# Natija:
# Title_num    0.28  ← EDA tasdiqlaдu!
# Sex_num      0.22  ← EDA tasdiqlaдu!
```

```
# Age_filled   0.17
# Log_Fare     0.14
# Pclass       0.10
# FamilySize   0.05
# IsAlone      0.03
# Embarked_num 0.01
```

---

11. Iris Dataset — Qisqacha Taqqoslash

```
from sklearn.datasets import load_iris

iris = load_iris()
df_iris = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
df_iris["species"] = iris.target_names[iris.target]

# Iris EDA — Pairplot:
sns.pairplot(df_iris, hue="species",
             palette="Set2", diag_kind="kde")
plt.show()

# Topilmalar:
# → setosa ANIQ ajralgan (chiziqli ajratiladi)
# → versicolor va virginica bir-biriga yaqin
# → petal_length va petal_width → Eng muhim xususiyatlar
# → sepal_width → Kamroq muhim
```

Iris vs Titanic:

```
Iris:
  → Oddiy, toza ma'lumot (bo'sh qiymat yo'q)
  → 3 sinf, 150 kuzatuv, 4 xususiyat
  → Klassifikatsiya o'rganish uchun ideal
  → Setosa ajralgan → Chiziqli usullar ishlaydi

Titanic:
  → Haqiqiy, murakkab ma'lumot
  → Bo'sh qiymatlar (Age 20%, Cabin 77%)
  → Feature Engineering talab qiladi
  → Domain bilimi muhim (kemaning tuzilishi)
  → ML pipeline o'rganish uchun ideal
```

---

12. Xulosa

```
EDA bosqichlari:
  1. Yuklash va dastlabki ko'rish (shape, info, head)
  2. Bo'sh qiymatlar va turlar
  3. Statistik tavsif (describe)
  4. Univariat → Har o'zgaruvchi alohida
  5. Bivariat  → O'zgaruvchi + Target
  6. Multivariat → Korrelyatsiya, pairplot
```

```
  7. Outlierlar → IQR, Z-score

Titanic Asosiy Topilmalar:
  → Jins: Eng kuchli prediktor (Ayol 74% vs Erkak 19%)
  → Sinf: Muhim (1-sinf 63% vs 3-sinf 24%)
  → Yosh: Bolalar ko'proq omon qolgan
  → Unvon: Jins + Yosh kombinatsiyasi (kuchli!)
  → Oila: U-shakl (kichik oila afzal)

EDA → Feature Engineering → Model:
  EDA topilmalari → Yangi xususiyatlar yaratish
  Yangi xususiyatlar → Model aniqligini oshirish
  Feature Importance → EDA ni tasdiqlash

Asosiy Qoida:
  ✅ EDA → Model emas, tushunish uchun
  ✅ Har grafik → Savol beradi, javob topadi
  ✅ Domain bilimi + Statistika = Yaxshi EDA
```

> 💡 **Eslab qol:** EDA — "Ma'lumot bilan suhbat". Har grafik bir savol, har topilma yangi savol tug'diradi. Titanic da "Ayollar birinchi" qoidasi raqamlarda tasdiqlandi — bu EDA ning kuchi! Model qurishdan oldin ma'lumotni **yaxshi tushungan** data scientist, tushunmagan data scientistdan doim **ustun** keladi.

🔴 **SAVOL 38:**

`IMBALANCED DATASET` MUAMMOSI: BIR SINFNING BOSHQASIDAN ANCHA KO'P BO'LISHI. `OVERSAMPLING` (`SMOTE`), `UNDERSAMPLING` VA `CLASS_WEIGHT` PARAMETRLARI YORDAMIDA YECHISH USULLARI.

---

1. Imbalanced Dataset — Muammo Nima?

```
Balanced dataset:
  Sinf 0: 500 ta  (50%)
  Sinf 1: 500 ta  (50%)
  → Model ikkalasini teng o'rganadi

Imbalanced dataset:
  Sinf 0: 950 ta  (95%)  ← Majority class
  Sinf 1:  50 ta   (5%)  ← Minority class
  → Model asosan Sinf 0 ni o'rganadi!
```

Nima uchun muammo:

```
"Ahmoq model" yondashuvi:
  Hammani Sinf 0 de → Accuracy = 95%!
  Lekin: Birorta Sinf 1 ni topmadi → Foydasiz!

Hayotiy misol:
  Saraton skriningi:
    99% sog', 1% kasal
```

```
    "Hammasi sog'" → 99% Accuracy
    Lekin 1% kasallar o'tkazib yuborildi → HALOKATLI!

  Firibgarlik aniqlash:
    99.9% normal, 0.1% firibgarlik
    "Hammasi normal" → 99.9% Accuracy
    Lekin firibgarlik aniqlanmadi → BANK ZARARI!
```

Qaysi sohalarda keng tarqalgan:

```
→ Tibbiy tashxis (kasal/sog')
→ Firibgarlik aniqlash (fraud detection)
→ Nosozlik aniqlash (fault detection)
→ Spam filtri (spam/ham)
→ Kiberxavfsizlik (hujum/normal)
→ Qarzni qaytarmaslik bashorati
→ Noyob kasalliklar
```

---

2. Imbalanced da Accuracy Aldamchi

```
Muammo ko'rsatish:

n=1000: Sinf 0 = 950, Sinf 1 = 50

Model A — "Ahmoq":
  Barcha 1000 ni Sinf 0 deydi
  Accuracy = 950/1000 = 95% ← Ajoyib ko'rinadi!
  Recall(Sinf 1) = 0/50 = 0% ← Hech narsa topilmadi!

Model B — "Yaxshi":
  850 ta Sinf 0 ni to'g'ri topdi
  40 ta Sinf 1 ni to'g'ri topdi
  Accuracy = 890/1000 = 89% ← Pastroq!
  Recall(Sinf 1) = 40/50 = 80% ← Ancha yaxshi!

Xulosa:
  Model A Accuracy bo'yicha yaxshi → ALDAMCHI
  Model B amalda yaxshi → FOYДALI

To'g'ri metrikalar:
  → Precision, Recall, F1-score
  → ROC-AUC
  → PR-AUC (Precision-Recall AUC) ← Imbalanced uchun eng yaxshi
  → Matthews Correlation Coefficient (MCC)
```

---

3. Yechim 1 — Class Weight (Eng Oson Usul)

G'oya: Minority class xatosiga ko'proq jarima berish. Model minority classga ko'proq e'tibor beradi.

```
Oddiy o'rgatish:
  Sinf 0 xatosi: 1 birlik jarima
```

```
    Sinf 1 xatosi: 1 birlik jarima
    → Model ko'p Sinf 0 ko'rgani uchun uni afzal ko'radi

  Class weight bilan:
    Sinf 0 xatosi: 1 birlik jarima
    Sinf 1 xatosi: 19 birlik jarima  ← Katta jarima!
    → Model Sinf 1 ni o'tkazib yuborishdan qo'rqadi
```

Vazn hisoblash:

```
  Balanced vazn formulasi:
    w_i = n / (k × n_i)

    n   → Jami kuzatuvlar
    k   → Sinflar soni
    n_i → i-sinf kuzatuvlari soni

  Misol: n=1000, k=2, n₀=950, n₁=50
    w₀ = 1000 / (2 × 950) = 0.526
    w₁ = 1000 / (2 × 50)  = 10.0

    Sinf 1 xatosi Sinf 0 dan 19x ko'p jarima!
```

```python
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

# Usul 1 — "balanced" avtomatik:
model = LogisticRegression(class_weight="balanced")
model = RandomForestClassifier(class_weight="balanced")
model = SVC(class_weight="balanced")

# Usul 2 — Qo'lda belgilash:
model = LogisticRegression(
    class_weight={0: 1, 1: 10}  # Sinf 1 ga 10x ko'proq e'tibor
)

# Usul 3 — Sklearn hisoblash:
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

vaznlar = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)
vazn_dict = dict(enumerate(vaznlar))
print(vazn_dict)   # {0: 0.526, 1: 10.0}

model = RandomForestClassifier(class_weight=vazn_dict)
```

Afzallik va kamchiliklari:

```
  Afzallik:
    ✅ Juda oson — faqat parametr qo'shish
    ✅ Ma'lumot o'zgarmaydi
    ✅ Hisoblash samarali
```

```
    ✅ Overfitting xavfi yo'q

  Kamchilik:
    ❌ Barcha algoritmlar qo'llab-quvvatlamaydi
    ❌ Optimal vazn topish qiyin
    ❌ Ba'zi holatlarda yetarli emas
```

---

4. Yechim 2 — Undersampling

G'oya: Majority classdan kam namuna olish → Balansga keltirish.

```
  Oldin:           Keyin:
  Sinf 0: 950  →   Sinf 0: 50
  Sinf 1:  50  →   Sinf 1: 50
                   ──────────
                   Jami: 100 (endi teng!)
```

Nazariy asos:

```
  Random Undersampling:
    Majority classdan tasodifiy namunalar olib tashlanadi

  Muammo:
    950 → 50 ga kamaytirish → 900 ta qimmatli ma'lumot YO'QOLDI!
    Kichik dataset qoladi → Model kam o'rganadi

  Yechim — Aqlli Undersampling:
    Tomek Links:
      Ikki sinf chegarasida joylashgan Sinf 0 nuqtalarni o'chirish
      → Chegara tozalanadi → Model aniq ajratadi

    NearMiss:
      Sinf 1 ga eng yaqin Sinf 0 nuqtalarni saqlash
      → Eng muhim chegaraviy namunalar qoladи
```

```python
from imblearn.under_sampling import (
    RandomUnderSampler,
    TomekLinks,
    NearMiss,
    EditedNearestNeighbours
)

# Random Undersampling:
rus = RandomUnderSampler(
    sampling_strategy=0.5,  # Minority/Majority = 0.5 bo'lsin
    random_state=42
)
X_res, y_res = rus.fit_resample(X_train, y_train)
print(f"Oldin: {y_train.value_counts().to_dict()}")
print(f"Keyin: {pd.Series(y_res).value_counts().to_dict()}")

# Tomek Links — chegara tozalash:
tl = TomekLinks()
```

```
X_res, y_res = tl.fit_resample(X_train, y_train)
# Faqat chegaradagi Sinf 0 nuqtalar o'chiriladi
# Katta o'zgarish bo'lmaydi, lekin chegara aniq bo'ladi
```

Undersampling qachon ishlatish:

```
✅ Juda katta dataset (millionlab namuna)
   → 1000 ta o'chirish ahamiyatsiz
✅ Majority class takroriy/redundant ma'lumot
✅ Tez prototiplash
✅ Tomek Links — chegara tozalash uchun

❌ Kichik dataset → Ma'lumot yo'qolishi kritik
❌ Har Sinf 0 namunasi qimmatli bo'lsa
```

---

5. Yechim 3 — Oversampling (SMOTE)

SMOTE (Synthetic Minority Over-sampling Technique) — 2002, Chawla et al.

Nazariy asos — G'oya:

```
Oddiy Random Oversampling:
  Sinf 1 namunalarini NUSXA OLISH (takrorlash)
  → 50 → 950 ta (bir xil 50 namunadan)
  → Overfitting! Model shu 50 ni yodlaydi

SMOTE — Yangi SINTETIK namunalar YARATISH:

  1. Minority class nuqtasini tanlash (A)
  2. Uning k ta yaqin qo'shnisini topish
  3. Tasodifiy yaqin qo'shni tanlash (B)
  4. A va B orasida tasodifiy nuqta yaratish (C)

  C = A + λ × (B - A)
  λ ∈ [0, 1] — tasodifiy son

Vizual:
  A ●──────────────────● B
            ↑
        C = Yangi sintetik nuqta

  Bu yangi, haqiqiyga o'xshash, lekin yangi ma'lumot!
```

```
from imblearn.over_sampling import (
    SMOTE,
    ADASYN,
    BorderlineSMOTE,
    RandomOverSampler
)

# Asosiy SMOTE:
smote = SMOTE(
    sampling_strategy="minority",  # Minority ni oshir
    k_neighbors=5,                  # Yaqin qo'shnilar soni
    random_state=42
```

```
)
X_res, y_res = smote.fit_resample(X_train, y_train)

print(f"Oldin: {pd.Series(y_train).value_counts().to_dict()}")
# Oldin: {0: 950, 1: 50}
print(f"Keyin: {pd.Series(y_res).value_counts().to_dict()}")
# Keyin: {0: 950, 1: 950}  ← Teng!

# ADASYN — Adaptive Synthetic:
# Qiyin namunalar atrofida ko'proq sintetik nuqta
adasyn = ADASYN(
    sampling_strategy="minority",
    n_neighbors=5,
    random_state=42
)

# BorderlineSMOTE — Faqat chegara yaqinida:
bsmote = BorderlineSMOTE(
    sampling_strategy="minority",
    k_neighbors=5,
    kind="borderline-1",  # yoki "borderline-2"
    random_state=42
)
```

SMOTE turlari:

```
Oddiy SMOTE:
  → Barcha minority nuqtalar uchun sintetik yaratish
  → Oddiy, keng qo'llaniladi

Borderline SMOTE:
  → Faqat chegara yaqinidagi nuqtalar uchun
  → G'oya: Chegara yaqini muhimroq
  → Ko'pincha SMOTE dan yaxshiroq

ADASYN:
  → Qiyin ajratiluvchi nuqtalarga ko'proq e'tibor
  → Adaptive — qiyinlik darajasiga qarab moslashadi
  → Ba'zan eng yaxshi natija

SVM-SMOTE:
  → Support vektorlar yaqinida sintetik yaratish
  → SVM bilan birgalikda ishlatilganda yaxshi
```

---

6. SMOTE + Pipeline — To'g'ri Usul

Juda Muhim: SMOTE faqat **train** ma'lumotida qo'llanilishi kerak!

```
from imblearn.pipeline import Pipeline   # ← sklearn emas, imblearn!
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler

# imblearn Pipeline SMOTE ni qo'llab-quvvatlaydi:
```

```python
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("smote",  SMOTE(random_state=42)),     # ← Faqat train da!
    ("model",  RandomForestClassifier(
                n_estimators=100,
                random_state=42))
])

# Cross-validation:
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
skorlar = cross_val_score(
    pipe, X_train, y_train,
    cv=cv,
    scoring="f1"     # F1 — imbalanced uchun
)
print(f"F1: {skorlar.mean():.4f} ± {skorlar.std():.4f}")

# Nima uchun imblearn Pipeline?
# sklearn Pipeline → SMOTE ni qo'llab-quvvatlamaydi
# imblearn Pipeline → SMOTE ni har fold da faqat train ga qo'llaydi
```

Data Leakage xavfi:

❌ NOTO'G'RI:

```
smote = SMOTE()
X_res, y_res = smote.fit_resample(X, y)   # BUTUN dataset!
cross_val_score(model, X_res, y_res, cv=5)

Nima bo'ldi?
Test ma'lumotiga o'xshash sintetik nuqtalar
train da bo'lishi mumkin!
→ Model test ni "oldindan ko'rgan" → Optimistik natija
```

✅ TO'G'RI:

```
imblearn Pipeline → Har CV fold da
SMOTE faqat train ga qo'llaniladi → Xavfsiz!
```

---

7. Kombinatsiyalangan Yondashuv — SMOTE + Tomek

```python
from imblearn.combine import SMOTETomek, SMOTEENN

# SMOTE + Tomek Links:
# 1. SMOTE → Minority oversampling
# 2. Tomek → Chegara tozalash (ikki tomondan)
smote_tomek = SMOTETomek(
    smote=SMOTE(random_state=42),
    tomek=TomekLinks(),
    random_state=42
)

# SMOTE + ENN (Edited Nearest Neighbours):
# 1. SMOTE → Minority oversampling
# 2. ENN   → Noto'g'ri tasniflangan nuqtalarni o'chirish
smote_enn = SMOTEENN(random_state=42)
```

```python
X_res, y_res = smote_tomek.fit_resample(X_train, y_train)

# Nima uchun kombinatsiya?
# SMOTE: Ko'paytiradi → Ba'zan shovqin ham ko'payadi
# Tomek/ENN: Tozalaydi → Shovqin o'chiriladi
# Ikkalasi birga → Yanada sof trening set
```

---

8. Usullarni Taqqoslash — Amaliy Sinov

```python
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.linear_model import LogisticRegression
import pandas as pd

usullar = {
    "Hech narsa (baseline)": None,
    "Class weight":
        LogisticRegression(class_weight="balanced"),
    "Random Undersampling":
        Pipeline([("us", RandomUnderSampler()),
                  ("m",  LogisticRegression())]),
    "SMOTE":
        Pipeline([("sm", SMOTE()),
                  ("m",  LogisticRegression())]),
    "SMOTE+Tomek":
        Pipeline([("st", SMOTETomek()),
                  ("m",  LogisticRegression())]),
}

natijalar = []
for nom, model in usullar.items():
    if model is None:
        model = LogisticRegression()

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:,1]

    natijalar.append({
        "Usul": nom,
        "Accuracy": (y_pred == y_test).mean(),
        "F1":       f1_score(y_test, y_pred),
        "ROC-AUC":  roc_auc_score(y_test, y_prob)
    })

df_natija = pd.DataFrame(natijalar)
print(df_natija.sort_values("F1", ascending=False))

# Kutilgan natija:
# Baseline:           Accuracy=0.95, F1=0.12, AUC=0.72
# Class weight:       Accuracy=0.88, F1=0.61, AUC=0.91
# Undersampling:      Accuracy=0.82, F1=0.55, AUC=0.87
# SMOTE:              Accuracy=0.91, F1=0.68, AUC=0.93
# SMOTE+Tomek:        Accuracy=0.92, F1=0.71, AUC=0.94 ← Eng yaxshi
```

---

9. Qaysi Usulni Tanlash?

```
Holat                          → Tavsiya
──────────────────────────────────────────────────────
Tez yechim kerak               → class_weight="balanced"
Katta dataset                  → Undersampling
Kichik dataset                 → SMOTE
Shovqinli ma'lumot             → SMOTE + Tomek/ENN
Nochiziqli chegara             → BorderlineSMOTE / ADASYN
Eksperimental                  → Barcha usullarni solishtirib ko'r

Imbalance darajasi:
  80/20 (yengil)  → class_weight yetarli
  90/10 (o'rta)   → SMOTE yoki class_weight
  99/1  (kuchli)  → SMOTE + Kombinatsiya + Threshold tuning

Algoritm:
  Linear model  → class_weight + SMOTE
  Tree models   → class_weight (ko'pincha yetarli)
  Neural Net    → class_weight + Focal Loss
```

---

10. Threshold Tuning — Qo'shimcha Usul

```python
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

# Model ehtimolliklarini olish:
y_prob = model.predict_proba(X_test)[:,1]

# Turli threshold larda Precision-Recall:
precision, recall, thresholds = precision_recall_curve(
    y_test, y_prob
)

# F1 maksimal bo'lgan threshold:
f1_scores = 2 * precision * recall / (precision + recall + 1e-8)
optimal_idx = f1_scores.argmax()
optimal_threshold = thresholds[optimal_idx]

print(f"Optimal threshold: {optimal_threshold:.3f}")
print(f"F1: {f1_scores[optimal_idx]:.4f}")

# Yangi bashorat:
y_pred_optimal = (y_prob >= optimal_threshold).astype(int)

# PR curve grafik:
plt.plot(recall, precision, "b-", linewidth=2)
plt.scatter(recall[optimal_idx], precision[optimal_idx],
            color="red", s=100, label=f"Optimal t={optimal_threshold:.2f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.show()
```

---

11. Xulosa

```
Imbalanced Dataset muammosi:
  → Minority class o'rganilmaydi
  → Accuracy aldamchi metrika
  → F1, ROC-AUC, PR-AUC → To'g'ri metrikalar

Yechimlar:

Class Weight:
  class_weight="balanced"
  ✅ Eng oson, ma'lumot o'zgarmaydi
  ✅ Ko'p algoritmda mavjud

Undersampling:
  Majority class kamaytirish
  ✅ Katta dataset uchun
  ❌ Ma'lumot yo'qoladi

SMOTE (Oversampling):
  Sintetik minority namunalar yaratish
  ✅ Ma'lumot yo'qolmaydi, yangi yaratiladi
  ✅ Ko'pincha eng yaxshi natija
  ❌ Shovqin yaratishi mumkin

Kombinatsiya:
  SMOTE + Tomek → Ko'paytir + Tozala
  ✅ Ko'pincha eng yaxshi

Muhim Qoidalar:
  ✅ SMOTE → Faqat train da
  ✅ imblearn Pipeline → Data Leakage oldini olish
  ✅ F1 / AUC → Accuracy emas
  ✅ Threshold tuning → Qo'shimcha yaxshilash
```

> 💡 **Eslab qol:** Imbalanced problem da "95% Accuracy" — **muvaffaqiyat emas, muvaffaqiyatsizlik belgisi** bo'lishi mumkin! Har doim F1 va ROC-AUC ga qarang. SMOTE sintetik ma'lumot yaratadi — bu "aldash" emas, balki model uchun "mashq qilish imkoniyati" yaratishdir. Lekin test to'plamiga hech qachon SMOTE qo'llamang — bu haqiqiy dunyo sinovidir!

🔴 **SAVOL 39:**

`HYPERPARAMETER TUNING` NIMA? `GRIDSEARCHCV` VA `RANDOMIZEDSEARCHCV` FARQLARI, QIDIRUV STRATEGIYALARI VA HISOBLASH NARXI. `N_ESTIMATORS` VA `MAX_DEPTH` UCHUN AMALIY MISOL.

---

1. Parametr vs Giperparametr — Farq

```
Parametrlar (Model O'RGATISH PAYTIDA topiladi):
  → Model o'zi topadi
  → Ma'lumotdan o'rganiladi
  → Misollar:
    LinearRegression → β₀, β₁, β₂...
    RandomForest     → Har daraxtning bo'lish nuqtalari
    NeuralNetwork    → Vaznlar (w) va biaslar (b)

Giperparametrlar (O'rgatishdan OLDIN belgilanadi):
  → Biz belgilaymiz (qo'lda yoki avtomatik)
  → Ma'lumotdan o'rganilmaydi
  → Misollar:
    RandomForest → n_estimators, max_depth, min_samples_leaf
    LogisticReg  → C, penalty, solver
    KMeans       → n_clusters, init, max_iter
    SVM          → C, kernel, gamma

Giperparametr Tuning:
  "Qaysi giperparametrlar kombinatsiyasi
   eng yaxshi model beradi?" → Avtomatik topish
```

---

2. Nima Uchun Tuning Muhim?

```
RandomForest, n_estimators=10:
  CV Accuracy = 0.76

RandomForest, n_estimators=100, max_depth=5:
  CV Accuracy = 0.89

RandomForest, n_estimators=200, max_depth=10, min_samples_leaf=2:
  CV Accuracy = 0.93

Bir xil algoritm → Giperparametrlar farqi → 17% farq!

Analogiya:
  Ferrari + yomon sozlash = Daydatsundan sekin
  Daydatsu + yaxshi sozlash = Ferraridan tez (ba'zan!)
```

---

3. GridSearchCV — To'liq Qidiruv

G'oya: Berilgan barcha kombinatsiyalarni to'liq sinab ko'rish.

```
n_estimators = [50, 100, 200]
max_depth    = [3, 5, 10]

Barcha kombinatsiyalar:
  (50, 3)  (50, 5)  (50, 10)
  (100, 3) (100, 5) (100, 10)
  (200, 3) (200, 5) (200, 10)
```

```
      Jami: 3 × 3 = 9 kombinatsiya

    cv=5 bilan:
      9 kombinatsiya × 5 fold = 45 ta model o'rgatiladi!
```

Kombinatsiyalar soni tez oshadi:

```
    2 parametr, 3 ta qiymat:  3² = 9
    3 parametr, 3 ta qiymat:  3³ = 27
    4 parametr, 3 ta qiymat:  3⁴ = 81
    5 parametr, 4 ta qiymat:  4⁵ = 1,024
    6 parametr, 5 ta qiymat:  5⁶ = 15,625  ← Juda ko'p!

    cv=5 bilan: 15,625 × 5 = 78,125 model!
```

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model",  RandomForestClassifier(random_state=42))
])

# Parametrlar panjara:
params = {
    "model__n_estimators": [50, 100, 200],
    "model__max_depth":    [3, 5, 10, None],
    "model__min_samples_leaf": [1, 2, 5],
}
# 3 × 4 × 3 = 36 kombinatsiya, cv=5 → 180 model

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    pipe,
    params,
    cv=cv,
    scoring="f1",          # Optimallash metriki
    n_jobs=-1,             # Barcha CPU yadrolari
    verbose=1,             # Progress ko'rsatish
    refit=True,            # Eng yaxshi modelni qayta o'rgatish
    return_train_score=True
)

grid.fit(X_train, y_train)

print(f"Eng yaxshi parametrlar: {grid.best_params_}")
print(f"Eng yaxshi CV F1:       {grid.best_score_:.4f}")
print(f"Test F1:                {grid.score(X_test, y_test):.4f}")
```

Natijalarni tahlil qilish:

```python
import pandas as pd

# Barcha natijalar:
```

```
natijalar = pd.DataFrame(grid.cv_results_)

# Eng yaxshi 5 ta:
ustunlar = [
    "param_model__n_estimators",
    "param_model__max_depth",
    "param_model__min_samples_leaf",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]
print(natijalar[ustunlar].sort_values(
    "rank_test_score").head(5))
```

---

4. RandomizedSearchCV — Tasodifiy Qidiruv

G'oya: Barcha kombinatsiyalar emas, tasodifiy N ta kombinatsiya sinash.

```
GridSearch:
  Barcha 1000 kombinatsiya → 1000 × cv model

RandomizedSearch:
  n_iter=50 → Tasodifiy 50 kombinatsiya → 50 × cv model
  20x tez!

Paradoks:
  "Tasodifiy tanlash GridSearch dan yaxshiroq bo'lishi mumkin?"

  Ha! Nima uchun?
  → Giperparametrlar ko'pincha bir xil ahamiyatli emas
  → Bir parametr muhim, boshqasi kam muhim
  → GridSearch kam muhim parametrga ko'p vaqt sarflaydi
  → RandomSearch muhim parametrni ko'proq qamrab oladi
```

Vizual tushuntirish:

```
2 parametr: A (muhim) va B (kam muhim)

GridSearch:          RandomSearch:
B                    B
│  × × ×             │  ×   ×  ×
│  × × ×             │     ×
│  × × ×             │  ×    ×  ×
└──────────── A      └──────────── A
  3×3=9 nuqta          9 tasodifiy nuqta

GridSearch: A bo'yicha faqat 3 ta qiymat sinaldi
RandomSearch: A bo'yicha 9 ta har xil qiymat!

→ RandomSearch muhim parametr A ni yaxshiroq qamrab oldi!
```

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
```

```python
# Uzluksiz taqsimotlar ham ishlatish mumkin!
params_rand = {
    "model__n_estimators": randint(50, 500),      # 50-500 orasida
    "model__max_depth":    randint(3, 20),         # 3-20 orasida
    "model__min_samples_leaf": randint(1, 10),
    "model__max_features": uniform(0.3, 0.7),     # 0.3-1.0 orasida
    "model__min_samples_split": randint(2, 20),
}

rand_search = RandomizedSearchCV(
    pipe,
    params_rand,
    n_iter=100,            # 100 ta tasodifiy kombinatsiya
    cv=cv,
    scoring="f1",
    n_jobs=-1,
    verbose=1,
    random_state=42,       # Takrorlanuvchi natija
    refit=True
)

rand_search.fit(X_train, y_train)

print(f"Eng yaxshi: {rand_search.best_params_}")
print(f"F1: {rand_search.best_score_:.4f}")
```

---

5. GridSearch vs RandomizedSearch — Taqqoslash

Xususiyat	GridSearchCV	RandomizedSearchCV
Qidiruv turi	To'liq (exhaustive)	Tasodifiy (random)
Kombinatsiyalar	Barcha	n_iter ta
Hisoblash vaqti	Ko'p	Kam (n_iter bilan)
Uzluksiz qiymat	❌ Qiyinroq	✅ scipy.stats bilan
Kafolat	Panjara ichida globalga yaqin	Yo'q (tasodifiy)
Ko'p parametr	❌ Juda sekin	✅ Tez
Kam parametr	✅ To'liq	⚠️ Ortiqcha
n_iter=barcha	—	GridSearch ga teng
Takrorlanuvchilik	✅ random_state	✅ random_state

```
Qachon qaysi:
  GridSearch  → 2-3 parametr, 2-4 qiymat (kam kombinatsiya)
  RandomSearch→ 4+ parametr, keng diapazon (ko'p kombinatsiya)
```

---

6. n_estimators va max_depth — Nazariy Ta'sir

n_estimators (Daraxtlar soni):

```
Nima qiladi:
  RandomForest = n_estimators ta Decision Tree
  Ko'proq daraxt → Ko'proq ovoz → Barqarorroq bashorat
```

```
Qanday ta'sir qiladi:
  n_estimators=10:
    → Tez, lekin beqaror (Variance yuqori)
    → Har run da boshqa natija

  n_estimators=100:
    → O'rtacha, barqaror

  n_estimators=500:
    → Juda barqaror
    → Lekin 10 dan sekin ishlaydi

  n_estimators ∞ → Asimptotik yaxshilanish:
    100 → 500 → Ko'proq yaxshilanish
    500 → 1000 → Kam yaxshilanish
    1000 → 2000 → Deyarli farq yo'q!

Grafik:
  Skor
  1.0 |           ──────────── (Plateau)
      |         /
      |       /
      |     /
      |   /
      | /
  0.7 |/
      └──────────────────── n_estimators
         10   100  500  1000

  Plateau → Keyin n_estimators oshirish foydasiz!
  Optimal: Plateau boshlanadigan nuqta
```

max_depth (Daraxt chuqurligi):

```
Nima qiladi:
  Har daraxt qanchalik "chuqur" o'sadigan — chegaralaydi

  max_depth=1 (Decision Stump):
    → Faqat 1 ta savol → Juda sodda
    → Underfitting

  max_depth=3-5:
    → O'rtacha murakkablik
    → Ko'pincha yaxshi

  max_depth=10:
    → Murakkab naqshlar
    → Overfitting xavfi

  max_depth=None (Cheksiz):
    → Har barg 1 nuqta bo'lguncha o'sadi
    → Bitta daraxtda overfitting
    → RF da bagging tufayli kamroq muammo

Chuqurlik → Bias-Variance:
  Kichik depth → Yuqori Bias, Past Variance (Underfitting)
```

```
Katta depth  → Past Bias, Yuqori Variance (Overfitting)
Optimal      → Ikkalasi muvozanat nuqtasi
```

---

7. Amaliy Misol — To'liq Qidiruv

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    GridSearchCV, RandomizedSearchCV,
    StratifiedKFold, cross_val_score
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from scipy.stats import randint
import time

# Ma'lumot yaratish:
X, y = make_classification(
    n_samples=2000, n_features=20,
    n_informative=10, n_redundant=5,
    weights=[0.85, 0.15],   # Imbalanced
    random_state=42
)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    stratify=y, random_state=42
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("rf", RandomForestClassifier(
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# ── GridSearchCV ──────────────────────────────────────

grid_params = {
    "rf__n_estimators": [50, 100, 200],
    "rf__max_depth":    [5, 10, 15, None],
    "rf__min_samples_leaf": [1, 2, 5],
}
# 3 × 4 × 3 = 36 × 5 = 180 model

grid = GridSearchCV(
```

```
    pipe, grid_params,
    cv=cv, scoring="f1",
    n_jobs=-1, verbose=0
)

t_start = time.time()
grid.fit(X_train, y_train)
grid_vaqt = time.time() - t_start

print("=" * 50)
print("GridSearchCV natijalari:")
print(f"  Vaqt:           {grid_vaqt:.1f} soniya")
print(f"  Eng yaxshi F1:  {grid.best_score_:.4f}")
print(f"  n_estimators:   {grid.best_params_['rf__n_estimators']}")
print(f"  max_depth:      {grid.best_params_['rf__max_depth']}")
print(f"  min_samples:    {grid.best_params_['rf__min_samples_leaf']}")
print(f"  Test F1:        {grid.score(X_test, y_test):.4f}")

# ── RandomizedSearchCV ──────────────────────────

rand_params = {
    "rf__n_estimators": randint(50, 500),
    "rf__max_depth":    [5, 10, 15, 20, None],
    "rf__min_samples_leaf": randint(1, 10),
    "rf__max_features": ["sqrt", "log2", 0.3, 0.5, 0.7],
    "rf__min_samples_split": randint(2, 15),
}

rand = RandomizedSearchCV(
    pipe, rand_params,
    n_iter=60,          # 60 ta tasodifiy kombinatsiya
    cv=cv, scoring="f1",
    n_jobs=-1, verbose=0,
    random_state=42
)

t_start = time.time()
rand.fit(X_train, y_train)
rand_vaqt = time.time() - t_start

print("\nRandomizedSearchCV natijalari:")
print(f"  Vaqt:           {rand_vaqt:.1f} soniya")
print(f"  Eng yaxshi F1:  {rand.best_score_:.4f}")
print(f"  n_estimators:   {rand.best_params_['rf__n_estimators']}")
print(f"  max_depth:      {rand.best_params_['rf__max_depth']}")
print(f"  Test F1:        {rand.score(X_test, y_test):.4f}")
print(f"\nTezlik farqi: Grid {grid_vaqt/rand_vaqt:.1f}x sekin")
```

---

8. n_estimators Grafik Tahlili

```
# n_estimators ta'sirini vizual o'rganish:

n_est_qiymatlar = [10, 25, 50, 75, 100, 150, 200, 300, 500]
train_skorlar = []
test_skorlar  = []
```

```
for n in n_est_qiymatlar:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("rf", RandomForestClassifier(
            n_estimators=n,
            max_depth=10,
            class_weight="balanced",
            random_state=42
        ))
    ])
    model.fit(X_train, y_train)
    train_skorlar.append(model.score(X_train, y_train))
    test_skorlar.append(model.score(X_test, y_test))

plt.figure(figsize=(10, 5))
plt.plot(n_est_qiymatlar, train_skorlar, "b-o",
         label="Train", linewidth=2)
plt.plot(n_est_qiymatlar, test_skorlar, "r-o",
         label="Test", linewidth=2)
plt.xlabel("n_estimators")
plt.ylabel("Accuracy")
plt.title("n_estimators ta'siri")
plt.legend()
plt.grid(True, alpha=0.3)
plt.axvline(x=100, color="green",
            linestyle="--", label="Tavsiya: 100")
plt.show()
```

---

9. max_depth Grafik Tahlili

```
# max_depth ta'sirini o'rganish:

depth_qiymatlar = [1, 2, 3, 4, 5, 7, 10, 15, 20, None]
depth_labels    = [str(d) for d in depth_qiymatlar]
train_f1 = []
test_f1  = []

from sklearn.metrics import f1_score

for depth in depth_qiymatlar:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("rf", RandomForestClassifier(
            n_estimators=100,
            max_depth=depth,
            class_weight="balanced",
            random_state=42
        ))
    ])
    model.fit(X_train, y_train)
    train_f1.append(f1_score(y_train, model.predict(X_train)))
    test_f1.append(f1_score(y_test, model.predict(X_test)))

plt.figure(figsize=(10, 5))
```

```python
x = range(len(depth_qiymatlar))
plt.plot(x, train_f1, "b-o", label="Train F1", linewidth=2)
plt.plot(x, test_f1, "r-o", label="Test F1", linewidth=2)
plt.xticks(x, depth_labels)
plt.xlabel("max_depth")
plt.ylabel("F1 Score")
plt.title("max_depth → Bias-Variance Tradeoff")
plt.legend()
plt.grid(True, alpha=0.3)

# Optimal nuqtani belgilash:
optimal = test_f1.index(max(test_f1))
plt.axvline(x=optimal, color="green",
            linestyle="--",
            label=f"Optimal: {depth_labels[optimal]}")
plt.legend()
plt.show()
```

---

10. Optuna — Zamonaviy Giperparametr Tanlash

```python
# Bayesian Optimization — GridSearch/RandomSearch dan aqlliroq
import optuna

def maqsad_funksiya(trial):
    """Har trial — bir kombinatsiya sinovi"""

    # Giperparametrlarni taklif qilish:
    n_est  = trial.suggest_int("n_estimators", 50, 500)
    depth  = trial.suggest_int("max_depth", 3, 20)
    leaf   = trial.suggest_int("min_samples_leaf", 1, 10)
    feat   = trial.suggest_float("max_features", 0.3, 1.0)

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("rf", RandomForestClassifier(
            n_estimators=n_est,
            max_depth=depth,
            min_samples_leaf=leaf,
            max_features=feat,
            class_weight="balanced",
            random_state=42
        ))
    ])

    skorlar = cross_val_score(
        model, X_train, y_train,
        cv=cv, scoring="f1"
    )
    return skorlar.mean()

# Optimallashtirish:
study = optuna.create_study(direction="maximize")
study.optimize(maqsad_funksiya, n_trials=50, show_progress_bar=True)
```

```
print(f"Eng yaxshi F1: {study.best_value:.4f}")
print(f"Parametrlar:   {study.best_params}")
```

Optuna vs Grid/Random:

```
GridSearch:    Barcha nuqta → Teng e'tibor → Sekin
RandomSearch:  Tasodifiy nuqta → Ko'r → O'rtacha
Optuna:        Avvalgi natijadan o'rganadi → Aqlli
               "Bu yerda yaxshi, bu tomonga qaray"
               → Tez va aniq!
```

---

11. Xulosa

```
Giperparametr vs Parametr:
  Parametr  → Model o'rgatishda topiladi
  Giperpar. → Biz oldindan belgilaymiz

GridSearchCV:
  → Barcha kombinatsiyalar
  → n₁ × n₂ × n₃ × cv ta model
  → Kafolatlangan (panjara ichida optimal)
  → Kam parametrda ishlatish

RandomizedSearchCV:
  → n_iter ta tasodifiy kombinatsiya
  → Uzluksiz taqsimotlar (scipy.stats)
  → Ko'p parametrda samarali
  → Grid dan tez, ba'zan yaxshiroq

n_estimators:
  → Ko'proq = Barqarorroq, lekin sekinroq
  → Plateau topish → Optimal n_estimators
  → Odatda 100-300 yetarli

max_depth:
  → Kichik = Underfitting (Yuqori Bias)
  → Katta = Overfitting (Yuqori Variance)
  → Optimal = Cross-validation bilan topiladi

Tanlash qoidasi:
  ≤3 parametr, ≤4 qiymat → GridSearch
  4+ parametr             → RandomizedSearch
  Resurs ko'p             → Optuna (Bayesian)
```

> 💡 **Eslab qol:** Giperparametr tuning — "eng yaxshi sozlamalarni topish" emas, "ma'lumotga mos modelni topish". GridSearch "to'rni barcha daryoga tashlash", RandomSearch "ko'p daryoda baliq borligini tekshirish". Optuna esa "baliq ko'p yerda ko'proq urinish" — aqlли strategiya!

🔴 **SAVOL 40:**

REGRESSIYA MODELLARINI BAHOLASH METRIKALARI: `MAE`, `MSE`, `RMSE` VA `R²` NING AFZALLIKLARI VA KAMCHILIKLARI. UY NARXINI BASHORAT QILISH MODELIDA QAYSI METRIKANI TANLASH KERAK?

---

1. Nima Uchun Metrika Muhim?

```
Xato = Haqiqiy - Bashorat = yᵢ - ŷᵢ

Muammo: Xatolarni qanday YIG'ISH kerak?

Variant 1 — Shunchaki yig'ish:
  Σ(yᵢ - ŷᵢ) → Musbat va manfiy xatolar bekor bo'ladi!
  +100 va -100 → Yig'indi = 0 → "Xato yo'q" (YOLG'ON!)

Variant 2 — Absolyut qiymat:
  Σ|yᵢ - ŷᵢ| → MAE

Variant 3 — Kvadrat:
  Σ(yᵢ - ŷᵢ)² → MSE

Har usul — boshqa narsani o'lchaydi!
Har kontekst — boshqa metrika kerak!
```

---

2. MAE — Mean Absolute Error

```
Formula:
  MAE = (1/n) · Σ|yᵢ - ŷᵢ|

Hisoblash misoli:
  Haqiqiy:  [300, 250, 400, 180, 500] (ming so'm)
  Bashorat: [320, 230, 410, 200, 460]
  Xatolar:  [-20, +20, -10, -20, +40]
  |Xatolar|: [20,  20,  10,  20,  40]
  MAE = (20+20+10+20+40)/5 = 110/5 = 22 ming so'm

Talqin:
  "Model o'rtacha 22 ming so'mga adashadi"
  → Eng intuitiv, tushunarli metrika!
```

MAE xususiyatlari:

```
Matematik:
  L1 norma → Absolyut qiymat
  Differensiallanmaydi (0 da) → Gradient descent qiyin
  Lekin subgradient mavjud

Outlierga munosabat:
  Xato 10 → 10 jarima
  Xato 100 → 100 jarima (10x ko'p, 10x katta xato)
  Chiziqli jarima → Outlier haddan oshiq ta'sir qilmaydi
```

```
Birlik:
  Y so'mda → MAE so'mda
  Talqin oson: "O'rtacha X so'm xato"
```

---

3. MSE — Mean Squared Error

```
Formula:
  MSE = (1/n) · Σ(yᵢ - ŷᵢ)²

Hisoblash:
  Xatolar:  [-20, +20, -10, -20, +40]
  Xato²:    [400, 400, 100, 400, 1600]
  MSE = (400+400+100+400+1600)/5 = 2900/5 = 580 (ming so'm)²

Talqin qiyin!
  MSE = 580 (ming so'm)² → Bu nima degani?
  Birlik kvadratda → Intuitiv emas
```

MSE xususiyatlari:

```
Matematik:
  L2 norma → Kvadrat
  Hamma yerda differensiallanadi → Gradient descent uchun ideal
  OLS (chiziqli regressiya) → MSE ni minimallaydi

Outlierga munosabat:
  Xato 10  → 100 jarima
  Xato 100 → 10,000 jarima (100x ko'p, 10x katta xato)
  Kvadratik jarima → Katta xatolarga KUCHLI jarima

  Misol:
  Kichik xatolar: [1, 2, 1, 2, 1] → MSE=1.4
  Bitta katta:    [1, 1, 1, 1, 10] → MSE=20.8
  → Bitta outlier MSE ni 15x oshirdi!
```

---

4. RMSE — Root Mean Squared Error

```
Formula:
  RMSE = √MSE = √[(1/n) · Σ(yᵢ - ŷᵢ)²]

Hisoblash:
  RMSE = √580 ≈ 24.1 ming so'm

Talqin:
  "Model o'rtacha 24.1 ming so'mga adashadi"
  → MAE ga o'xshash talqin, lekin katta xatolarga sezgir

Nima uchun MSE emas RMSE?
  MSE = 580 (ming so'm)² → Tushunarsiz birlik
  RMSE = 24.1 ming so'm → Y bilan bir xil birlik → Tushunarli!
```

RMSE vs MAE — Farq:

```
Kichik, bir xil xatolar:
  Xatolar: [10, 10, 10, 10, 10]
  MAE  = 10
  RMSE = 10   ← Teng!

Bir katta, qolgani kichik:
  Xatolar: [1, 1, 1, 1, 50]
  MAE  = 10.8
  RMSE = 22.4  ← Kattaroq! (50 li xatoga kuchli jarima)

RMSE > MAE → Har doim!
RMSE - MAE katta → Katta xatolar (outlierlar) bor
RMSE ≈ MAE → Xatolar bir xil darajada
```

---

5. R^2 — Determinatsiya Koeffitsienti

```
Formula:
  SS_res = Σ(yᵢ - ŷᵢ)²    ← Model xatosi
  SS_tot = Σ(yᵢ - ȳ)²     ← Umumiy o'zgarish

  R² = 1 - SS_res/SS_tot

Intuitiv tushuncha:
  SS_tot → "Model yo'q bo'lsa, faqat o'rtachani bashorat qilsak"
  SS_res → "Bizning modelimizning xatosi"

  R² = 1 - (Model xatosi / Eng sodda model xatosi)

Talqin:
  R² = 0.85 → "Model Y dagi o'zgarishlarning 85% ini tushuntiradi"
  R² = 1.00 → Mukammal (amalda shubhali → Overfitting!)
  R² = 0.00 → Faqat o'rtachani bashorat qilish darajasida
  R² < 0.00 → O'rtachadan ham yomoni (juda yomon model!)
```

R^2 ning noyob xususiyati — Miqyossizlik:

```
MAE, MSE, RMSE → Birlikka bog'liq
  Uy narxi so'mda → RMSE so'mda
  Uy narxi dollarda → RMSE dollarda
  → Turli birlikdagi modellarni taqqoslab bo'lmaydi!

R² → 0 dan 1 gacha, birliksiz
  Uy narxi so'mda → R²=0.85
  Uy narxi dollarda → R²=0.85 (bir xil!)
  → Turli modellar, turli birliklar → Taqqoslash mumkin!
```

R^2 ning kamchiliklari:

```
Kamchilik 1 — Yangi o'zgaruvchi qo'shilsa R² hech qachon kamaymas:
  X₁ bilan: R²=0.75
  X₁, X₂ (tasodifiy) bilan: R²=0.76  ← Oshdi!
```

```
  X₁, X₂, X₃ (tasodifiy) bilan: R²=0.77 ← Yana oshdi!

  Yechim → Adjusted R²:
  R²_adj = 1 - (1-R²)·(n-1)/(n-p-1)
  p → O'zgaruvchilar soni
  Keraksiz o'zgaruvchi qo'shilsa → Adjusted R² KAMAYADI

Kamchilik 2 — Chiziqli bo'lmagan munosabatda past:
  Y = X² munosabat → Chiziqli model R²=0.60 (past ko'rinadi)
  Lekin polinom model R²=0.98
  → R² past = Model yomon, degani emas
     R² past = Bu MODEL yomon bo'lishi mumkin

Kamchilik 3 — Sohaga bog'liq talqin:
  Ijtimoiy fanlar: R²=0.4 → Yaxshi!
  Fizika:          R²=0.4 → Juda yomon!
```

---

6. To'liq Taqqoslash Jadvali

Xususiyat	MAE	MSE	RMSE	R^2
Formula	$\Sigma\|e\|/n$	$\Sigma e^2/n$	$\sqrt{(\Sigma e^2/n)}$	1-SS_res/SS_tot
Birlik	Y bilan bir xil	Y^2	Y bilan bir xil	Birliksiz
Talqin osonligi	✅ Oson	❌ Qiyin	✅ Oson	✅ % sifatida
Outlier ta'siri	⚠️ Chiziqli	❌ Kuchli	❌ Kuchli	❌ Kuchli
Differensiallash	⚠️ 0 da yo'q	✅ Har yerda	✅ Har yerda	✅
Modellar taqqoslash	⚠️ Bir birlikda	⚠️ Bir birlikda	⚠️ Bir birlikda	✅ Universal
Optimallash	✅ LAD regression	✅ OLS	✅ OLS	✅
Katta xato jarima	Chiziqli	Kvadratik	Kvadratik	Kvadratik

---

7. Uy Narxini Bashorat Qilish — Qaysi Metrika?

```
Kontekst:
  Uy narxi: 50 million — 5 milliard so'm oralig'ida
  Ma'lumot: Ko'p outlier (luxury uylar, noyob manzillar)
  Foydalanuvchilar: Xaridor, sotuvchi, rieltor, bank
```

Asosiy tavsiya — RMSE + MAE + R^2 kombinatsiyasi:

RMSE — Asosiy metrika:

```
Nima uchun RMSE:
  1. Katta xatolar = Katta muammo:
     100 million so'mga adashish → Xaridor zarar ko'radi
     RMSE katta xatolarga ko'proq e'tibor beradi ✅

  2. Matematik qulay:
```

```
   Gradient descent → RMSE (MSE) ni minimallash
   OLS → MSE ni minimallaydi

3. Y bilan bir xil birlik:
   RMSE = 50 million so'm
   → "Model o'rtacha 50 million so'mga adashadi"

4. Bank/moliya uchun:
   Garovga olish, kredit berish → Katta xato = Katta risk
   RMSE katta xatolarni ko'rsatadi → Risk baholash uchun ✅
```

MAE — Qo'shimcha metrika:

```
Nima uchun MAE ham kerak:
  1. Median xatoni ko'rsatadi (outlierga chidamli):
     Luxury uy (10 milliard) → RMSE ni "og'diradi"
     MAE o'rtacha xaridor uchun real xatoni ko'rsatadi

  2. RMSE vs MAE taqqoslash → Outlier borligini aniqlash:
     RMSE >> MAE → Katta outlierlar bor (luxury uylar?)
     RMSE ≈ MAE → Xatolar bir tekis taqsimlangan

  3. Rieltor uchun:
     "O'rtacha qancha adashadi?" → MAE ko'proq ma'noli
```

R^2 — Tushuntiruvchi metrika:

```
Nima uchun R²:
  1. "Model qanchalik yaxshi?" — Foizda tushuntiriladi:
     R²=0.87 → "Model narx o'zgarishlarining 87% ini tushuntiradi"
     Xaridorga tushuntirish oson

  2. Modellar taqqoslash:
     Linear R²=0.75, RandomForest R²=0.89
     → RandomForest ancha yaxshi

  3. Adjusted R² → Ortiqcha o'zgaruvchilarni aniqlash
```

Qachon MSE:

```
Bank kredit modeli:
  Uy narxini MSE → Portfel riskini hisoblash
  Matematik optimallash → MSE differensiallash uchun qulay
  Risk menejmenti → Dispersiya (MSE) muhim ko'rsatkich
```

---

8. Sklearn da Hisoblash

```python
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error
)
```

```python
import numpy as np

def modelni_baholash(y_haqiqiy, y_bashorat, nom="Model"):
    mae  = mean_absolute_error(y_haqiqiy, y_bashorat)
    mse  = mean_squared_error(y_haqiqiy, y_bashorat)
    rmse = np.sqrt(mse)
    r2   = r2_score(y_haqiqiy, y_bashorat)
    mape = mean_absolute_percentage_error(y_haqiqiy, y_bashorat)

    print(f"\n{'='*40}")
    print(f"  {nom}")
    print(f"{'='*40}")
    print(f"  MAE:  {mae:>15,.0f} so'm")
    print(f"  RMSE: {rmse:>15,.0f} so'm")
    print(f"  R²:   {r2:>15.4f}")
    print(f"  MAPE: {mape*100:>14.2f}%")
    print(f"\n  RMSE/MAE nisbat: {rmse/mae:.2f}")
    if rmse/mae > 1.5:
        print("  ⚠️ Katta outlierlar bor!")
    else:
        print("  ✅ Xatolar bir tekis")

    return {"MAE": mae, "RMSE": rmse, "R2": r2}

# Misol:
y_haqiqiy  = np.array([500, 750, 1200, 300, 2000,
                       450, 650, 900, 1500, 350])
y_baseline = np.full_like(y_haqiqiy, y_haqiqiy.mean())
y_pred_lr  = np.array([520, 730, 1150, 320, 1900,
                       470, 680, 870, 1450, 370])
y_pred_rf  = np.array([510, 755, 1190, 305, 1980,
                       455, 660, 895, 1480, 355])

modelni_baholash(y_haqiqiy, y_baseline, "Baseline (O'rtacha)")
modelni_baholash(y_haqiqiy, y_pred_lr,  "Chiziqli Regressiya")
modelni_baholash(y_haqiqiy, y_pred_rf,  "Random Forest")
```

---

9. MAPE — Foiz Xatosi

```
Formula:
  MAPE = (1/n) · Σ|yᵢ - ŷᵢ| / |yᵢ| × 100%

Misol:
  Uy narxi: 500 million, Bashorat: 550 million
  MAPE ulushi: |500-550|/500 = 10%

  Uy narxi: 2000 million, Bashorat: 2050 million
  MAPE ulushi: |2000-2050|/2000 = 2.5%

  O'rtacha MAPE = (10% + 2.5%) / 2 = 6.25%

Talqin:
  "Model o'rtacha 6.25% ga adashadi"
  → Narxdan mustaqil → Arzon va qimmat uylar teng baholanadi
```

```
Kamchilik:
  yᵢ = 0 → Bo'linish muammosi!
  Kichik qiymatlar uchun noto'g'ri (asimmetrik)
  MAPE = 50% → +50% yoki -33.3% bo'lishi mumkin
```

---

10. Xulosa

```
Metrikalar:
  MAE  → O'rtacha mutlaq xato, outlierga chidamli
         Talqin: "O'rtacha X so'm adashadi"

  MSE  → Kvadratik xato, matematik qulay
         Talqin: Qiyin (birlik = Y²)

  RMSE → MSE ildizi, Y bilan bir xil birlik
         Talqin: MAE kabi, lekin katta xatolarga sezgir

  R²   → 0-1 orasida, birliksiz, universal taqqoslash
         Talqin: "Y ning X% ni tushuntiradi"

Uy narxi bashoratida:
  Asosiy → RMSE
    ✅ Katta xatolarga ko'proq jarima
    ✅ Y bilan bir xil birlik
    ✅ Bank/risk baholash uchun mos

  Qo'shimcha → MAE
    ✅ Outlier ta'sirini ko'rish
    ✅ RMSE/MAE > 1.5 → Outlierlar bor

  Universal → R²
    ✅ Foizda tushunarli
    ✅ Modellar taqqoslash

  Kombinatsiya:
    RMSE + MAE + R² → To'liq rasm
    Faqat bitta metrika → ALDAMCHI!
```

> 💡 **Eslab qol:** RMSE va MAE ni birga ishlatish — bu "stethoskop va EKG" kabi. MAE umumiy holatni, RMSE katta muammolarni ko'rsatadi. Uy narxida **RMSE >> MAE** bo'lsa — modelingiz ba'zi uylar uchun juda yomoni degani. Bu luxury uylar yoki anomal narxlar bo'lishi mumkin — amaliy qaror qabul qilishda bu farqni bilish juda muhim!